

ContextFlow: 基于重放与上下文流水线架构的 量化卷积神经网络加速器

郝嘉恒

(东南大学吴健雄学院 无锡 214125)

摘要 在资源受限的 FPGA 平台上, 固定规模阵列通常无法一次完成整个卷积层的计算, 因此需要将卷积拆分为多个分块, 由阵列分时执行。现有量化 CNN 加速方法虽可通过输入驻留和访存-计算重叠减少数据搬运与等待, 但在一个分块需要多次阵列计算时, 同一输入仍可能被重复读取或转换, 相邻计算之间也可能因数据准备而出现空闲。针对这些问题, 本文提出 CONTEXTFLOW。片上向量生成与重放机制将每个紧凑 HWC 分块从 DDR 读取一次, 在片上形成阵列可直接消费的向量序列, 并供不同输出通道块重复使用; 面向折叠执行的上下文流水机制则在满足依赖的条件下, 并行推进相邻计算的输入与权重准备、阵列计算、部分和反馈及结果提交。基于 XCK26/KV260 平台, 采用 W8A8 量化和 INT32 累加的完整双尺度 YOLOv3-tiny 推理系统在 200 MHz 下达到 28.618 FPS、165.588 GOPS 及 71.87% 的有效阵列利用率。消融实验中, 片上向量生成与重放使输入访存流量降低 19.67 倍, 常驻推理获得 2.020 倍加速; 加入上下文流水后, 常驻推理延迟进一步降至 34.943 ms。

关键词 卷积神经网络; 现场可编程门阵列; 脉动阵列; 片上向量重放; 折叠执行; 上下文流水

ContextFlow: A Replay-Enabled Context-Pipelined Accelerator for Quantized Convolutional Neural Networks

Jiaheng Hao

(Chien-Shiung Wu College, Southeast University, Wuxi 214125)

Abstract On a resource-constrained FPGA, a fixed-size array usually cannot complete an entire convolutional layer of a quantized CNN in a single execution, so the layer must be divided into multiple tiles and processed over time. Input-stationary and memory-compute overlap can reduce data movement and waiting, but when a tile requires multiple array executions, the same input may still be read or transformed repeatedly, and adjacent executions may remain idle while data are prepared. This paper presents CONTEXTFLOW. Its on-chip vector generation and replay mechanism reads each compact HWC tile once from DDR, forms an array-ready vector sequence on chip, and reuses it across output-channel blocks. Context pipelining for folded execution overlaps input and weight preparation, array computation, partial-sum feedback, and result retirement across adjacent executions when dependencies permit. On the XCK26/KV260 platform, the complete dual-head YOLOv3-tiny inference system with per-tensor W8A8 quantization and INT32 accumulation achieves 28.618 FPS, 165.588 GOPS, and 71.87% effective array utilization at 200 MHz. Ablation results show that on-chip vector generation and replay reduces input memory traffic by 19.67 $\times$ and accelerates resident inference by 2.020 $\times$; adding context pipelining further reduces resident inference latency to 34.943 ms.

Keywords convolutional neural network; field-programmable gate array; systolic array; on-chip vector replay; folded execution; context pipelining

1 引言

随着目标检测等视觉任务不断向边缘设备迁移, CNN 推理已成为边缘视觉系统中最主要的计算负载之一。FPGA 能够围绕数据位宽和网络结构定制并行资源,因而被广泛用于构建高效 CNN 推理加速器^[1-3]。在边缘多处理器片上系统 (MPSoC) 中,固定规模的脉动阵列通常承担 MAC,处理器与片外存储器负责网络调度和张量搬运。阵列峰值算力只是性能上限;数据能否持续供给以及相邻计算能否连续衔接,决定了系统实际维持的有效算力。

当单个输出所需的乘加数量或输出通道数超过阵列的并行能力时,一个卷积层需要拆分为多次阵列计算。本文将这种映射方式称为折叠执行。已有加速器通过数据流映射、输入驻留、片上表示转换、部分和缓冲以及异步访存来降低数据移动并重叠传输与计算^[4-9]。这些设计大多以输入分块、阵列流水或访存/执行命令为组织单位,而固定阵列在折叠映射下反复处理的是一个输入片段与一个输出通道块的组合。输出块切换时,权重发生变化但输入保持不变;计算轮次推进时,输入片段随之变化,并携带前一轮的部分和依赖。若复用范围和执行交接仍止于较粗粒度,阵列输入向量会在输出块之间重复取得或生成,下一次已经就绪的折叠执行也要等待当前执行完全收尾。

在本文使用的 YOLOv3-tiny 中,深层卷积 m13 的每个输出需要累加 4608 个乘积。18 行阵列每轮处理 18 个乘加项,因此完成一次输出需要 256 轮计算;1024 个输出通道又形成 32 个输出块,使一个空间分块需要 8192 次折叠执行。沿输出块方向,阵列输入向量的重复读取或生成会抵消算法层面的输入复用;沿执行序列方向,若地址准备、权重装载、部分和返回与结果提交在每个边界串行推进,短暂停顿便会在数千次执行中累积。本文将相邻阵列执行的两次有效发射之间没有有效计算的周期称为上下文边界气泡 (context-boundary bubble)。

为减少上述重复输入和执行间等待,本文提出 CONTEXTFLOW,以单次阵列计算为基本调度单位,

统一组织输入数据的片上复用和相邻计算的流水交接。紧凑 HWC 分块在片上转换为阵列可直接消费的向量序列,并供不同输出通道块重放;每次阵列计算被组织为一个上下文,使相邻上下文的准备、执行和退休阶段能够在依赖满足时交叠。图 1 展示了跨输出块的输入重复、相邻上下文的边界气泡及相应的处理方式。

本文围绕这一粒度实现了片上向量生成与重放机制:紧凑 HWC 分块只读取一次,随后在片上生成按像素和计算轮次排列的阵列输入向量,并由不同输出通道块共享。片外输入流量模型与完整网络的 DMA 字节计数用于评估其流量和性能变化。在同一执行模型中,面向折叠执行的上下文流水机制将地址准备、权重预载、阵列计算、部分和反馈及结果提交组织为可重叠阶段,并通过下一轮预取、计算期预取和上下文直接切换缩短相邻执行之间的气泡。

本文在 XCK26/KV260 上实现了完整的双尺度 YOLOv3-tiny 推理系统,13 个卷积层由同一固定阵列执行。在 200 MHz 下,系统达到 28.618 FPS、165.588 GOPS 和 71.87% 的有效阵列利用率。片上向量生成与重放将完整网络输入流量降低 19.67 倍,并使常驻推理延迟由 156.457 ms 降至 77.445 ms;加入面向折叠执行的上下文流水后,延迟进一步降至 34.943 ms。

下文先说明固定阵列上的折叠映射及其复用、交接开销,再介绍 CONTEXTFLOW 的执行模型与实现机制;随后给出实验评估,并讨论适用范围和相关工作。

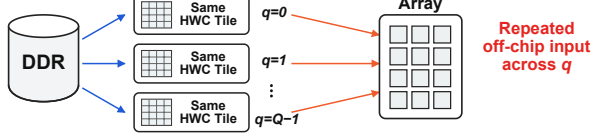
2 背景与折叠映射

2.1 量化卷积

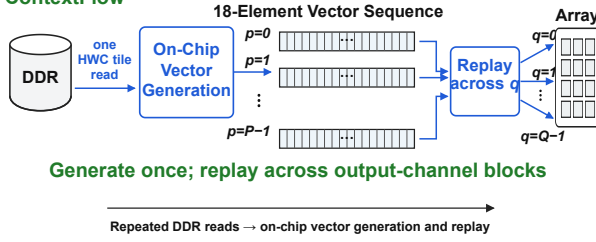
本文采用静态逐张量 W8A8 量化,各层使用独立的量化参数。部署前将 BatchNorm 参数折叠到卷积权重和偏置中;激活采用非对称仿射量化,权重采用对称量化。输入按激活零点中心化后参与乘加,乘积和偏置在 INT32 域累加;累加结果经定点乘法

1 Data-Locality Gap

Serialized q -Block Execution



ContextFlow



2 Execution-Continuity Gap

Serialized Context Boundary



Context-Pipelined Folded Execution

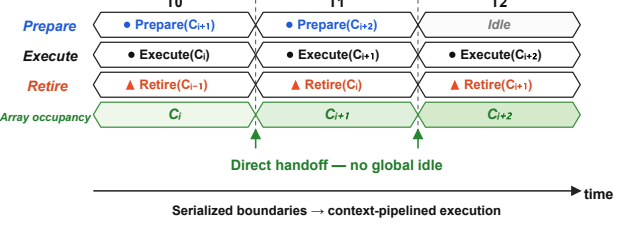


图 1 折叠执行中的输入重复与上下文边界气泡。片上向量生成与重放减少跨输出通道块的重复输入，上下文流水通过预取、部分和反馈与直接切换缩短相邻执行边界。

和移位完成重定标，再由查找表完成激活与输出量化。对输出位置 (y, x) 和输出通道 c_o ，卷积写为

$$Y[y, x, c_o] = b[c_o] + \sum_{k=0}^{K-1} X_{y,x}[k]W[c_o, k], \quad (1)$$

其中 $K = k_h k_w C_{in}$ 表示计算一个输出元素需要累加的乘积数。 X 、 Y 和累加过程中的中间结果分别记为 IFM、OFM 和 PSUM。若一次输出需要多轮计算，非首轮从上一轮 PSUM 继续累加，末轮才完成重定标和输出。

2.2 数据复用与传输重叠基础

卷积中的输入、权重和 PSUM 具有不同的复用方向，数据流与存储层次决定复用发生在处理单元、片上缓存还是片外存储器^[4,10]。容量有限的嵌入式 FPGA 通常先将特征图和权重划分为可驻留分块，再通过循环次序选择驻留对象。输入驻留、片上图像到列转换（image-to-column, im2col）、部分和缓冲和乒乓传输已经分别用于减少中间张量、支持分段累加并隐藏部分访存延迟^[5-8]。解耦的 LOAD、STORE 与 EXECUTE 队列还可以显式表达跨命令依赖^[9]。这些机制构成本文的数据通路和控制基础；后文关注的是它们在固定阵列折叠序列中的复用范围和交接粒度。

表 1 核心符号及其含义

符号	含义
K	单个输出元素需要累加的乘积数 $k_h k_w C_{in}$
R	阵列每轮并行处理的乘加项数，本文为 18
C_T	阵列一次并行产生的输出数，本文为 32
P_l	层 l 完成一次输出所需的计算轮数，等于 $\lceil K_l/R \rceil$
Q_l	层 l 的输出通道块数，等于 $\lceil C_{out,l}/C_T \rceil$

2.3 固定规模阵列上的折叠执行粒度

本文目标阵列每次并行处理至多 $R = 18$ 个乘加项，物理上由 16 列双输出处理通路并行产生 $C_T = 32$ 个输出。层 l 完成一次输出所需的计算轮数和输出通道块数分别为

$$P_l = \left\lceil \frac{K_l}{R} \right\rceil, \quad Q_l = \left\lceil \frac{C_{out,l}}{C_T} \right\rceil. \quad (2)$$

其中， P_l 轮计算依次处理构成一个输出的不同输入片段，轮次编号记为 p ；同一输出块内各轮通过 PSUM 顺序关联。不同输出块的编号记为 q ，它们使用不同权重，但共享相同空间位置的输入。

如图 2 和表 2 所示，浅层 m0 只需两轮计算，

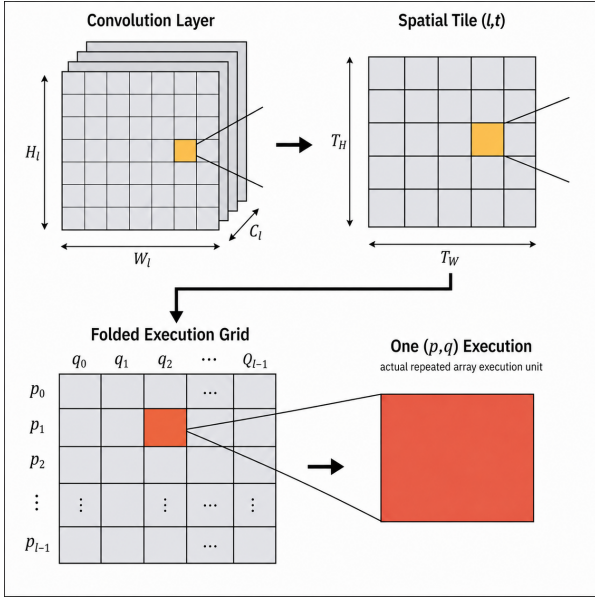


图 2 固定规模阵列上的折叠映射。从卷积层选取空间分块后，阵列沿计算轮次 p 和输出通道块 q 形成执行网格；其中一个 (p, q) 单元对应一次实际阵列执行。

表 2 代表层的折叠规模。执行次数按单个空间分块计算。

层	K	P_l	C_{out}	Q_l	$P_l Q_l$
m0	27	2	16	1	2
m13	4608	256	1024	32	8192
m19	3456	192	256	8	1536
P4 检测头	256	15	255	8	120
P5 检测头	512	29	255	8	232

而 m13 和 m19 在同一空间分块内分别产生 8192 和 1536 次折叠执行。固定规模阵列由此把一个卷积层展开为长执行序列：沿 q 方向，权重随输出块改变而输入保持不变；沿 p 方向，不同输入片段通过 PSUM 顺序关联。一次 (p, q) 组合才是阵列反复接收数据并完成部分累加的实际执行单位。

输入分块驻留的范围未必覆盖由其派生的阵列向量，而命令级重叠也未规定相邻 (p, q) 执行之间的权重、PSUM 和阵列执行权如何交接。下一节据此分析折叠序列中的输入复用和执行边界开销。

3 研究动机：折叠执行中的复用与交接开销

固定阵列把卷积展开为沿 p 和 q 遍历的执行序列，但这种映射本身并不能保证输入得到充分复用，

也不能保证相邻计算连续衔接。图 1 将这两个问题直观地并列展示：左侧说明同一 HWC 分块在输出通道块之间可能被重复读取或转换，右侧说明串行的准备、计算和提交会在相邻执行之间留下气泡。输入驻留、片上展开、PSUM 缓冲和异步访存仍然是数据通路基础，关键在于把它们的作用范围进一步落实到阵列反复执行的 (p, q) 组合。下面分别从输入流量和执行间隔出发说明这两类开销，并由此引出 CONTEXTFLOW 的设计选择。

3.1 跨输出通道块的输入重复

紧凑输入、局部展开结果和阵列输入流是不同的数据表示。本文的系统边界保存紧凑 HWC 分块，而阵列按 p 消费定长向量。令 $B_{X,p}$ 表示空间分块中第 p 轮计算所需的紧凑输入字节。当每个输出通道块都重新取得这些输入片段，而重放机制使输入只进入片上一次并由全部输出块共享时，两种流量分别为

$$D_{repeat} = Q_l \sum_{p=0}^{P_l-1} B_{X,p}, \quad (3)$$

$$D_{replay} = \sum_{p=0}^{P_l-1} B_{X,p} \approx B_X.$$

不同 p 消费不同通道片段，其字节数累加为一个输入分块；不同 q 只改变权重，理论上可以复用同一组输入。因此，若阵列输入表示不能跨 q 保持，图 1 左侧所示的重复便会使流量随输出块数增长；理想流量降低上限主要由 Q_l 决定。实际流量还受到边界行复用、空间分块数量和 DMA 突发对齐影响，式(3)用于刻画复用机会，而不预设实测降低倍数。

软件端显式展开会引入额外中间张量和处理器开销；将 HWC 分块缓存到片上，也只保留了紧凑表示。为了让输入局部性覆盖输出块遍历，阵列可直接消费的表示需要在片上按 p 生成，并在全部 q 遍历期间保持有效。

3.2 相邻执行之间的边界等待

双缓冲和异步队列可以并行推进访存与计算，但同一空间分块内的相邻 (p, q) 执行还要交接权重、PSUM 和阵列执行权。设 a_i^{last} 为第 i 次折叠执行最

后一次有效阵列发射周期, a_{i+1}^{first} 为下一次执行的第一次有效发射周期, 则边界气泡及其在单个空间分块内的累积可写为

$$G_i = \max(0, a_{i+1}^{first} - a_i^{last} - 1), \quad G_{tile} = \sum_{i=0}^{P_t Q_t - 2} G_i. \quad (4)$$

输入地址准备、权重装载、PSUM 返回和结果提交若由同一串行状态机依次推进, 都会表现为 G_i , 对应图 1 右侧的上下文边界气泡。必须保持顺序的是阵列执行权、同一像素的 PSUM 累加关系和外部输出次序; 下一次执行的地址与权重准备, 以及前一次执行的数据提交, 并不都依赖当前阵列计算结束。单次气泡可能很短, 但一个空间分块内最多经历 $P_t Q_t - 1$ 个相邻边界, 累计开销会随折叠序列增长。

上述分析表明, 输入重复与边界等待虽然发生在不同数据路径上, 却都源于复用和调度没有覆盖到单次阵列执行。为此, 本文把一次 (l, t, q, p) 折叠执行定义为共同的控制单位: 下一上下文可以准备, 当前上下文占用阵列执行, 前一上下文则完成退休; PSUM 和资源依赖决定阵列何时直接交接。该粒度既承载按 p 生成、跨 q 保持有效的输入向量, 也连接相邻执行的准备、计算和提交。

由此, ContextFlow 采用片上向量生成与重放机制消除跨输出块的重复输入, 并以面向折叠执行的上下文流水机制 (context pipelining for folded execution) 缩短相邻上下文之间的等待。后续实验以输入 DMA 流量衡量前者, 并以完整网络时延和有效阵列利用率衡量后者的综合效果。下一节给出执行上下文的统一描述和总体架构。

4 ContextFlow 总体架构与执行模型

4.1 系统总体架构

图 3 展示 ContextFlow 所在的异构推理系统。处理 PS 通过 DDR 访问网络张量, PL 完成全部卷积。输入、权重、偏置和输出分别使用 DMA 通道, 并通过 AXI 连接片外存储器。外部张量始终保持紧凑 HWC 布局, 卷积窗口展开、分段累加和执行调度封

表 3 上下文三级生命周期及其主要工作

阶段	主要工作	关键资源
准备	形成输入、权重和上下文描述	向量缓存区、备用权重槽
执行	发射有效向量并完成当前轮累加	阵列、PSUM 读槽
退休	提交 PSUM 或 OFM 并释放身份	PSUM 写槽、输出队列

装在 PL 内部。

片上存储按容量和访问宽度分工: URAM 保存生成后的输入向量, BRAM 保存行缓存、权重和 PSUM, LUT 与寄存器实现控制队列和流水切片。阵列乘法映射到 DSP。相互独立的数据通路使输入转换、权重准备、阵列执行和结果提交能够并发推进。

4.2 执行上下文与三级生命周期

本文把由固定网络层、空间分块、输出通道块和计算轮次确定的一次阵列执行称为 **执行上下文 (execution context)**, 记为

$$C_i = (l, t, q, p, f_i, r_i), \quad (5)$$

其中 l 为网络层, t 为空间分块, q 为输出通道块, p 为计算轮次编号, f_i 和 r_i 标记首轮与末轮。沿 p 方向, 上下文通过 PSUM 顺序关联; 沿 q 方向, 上下文共享输入而使用不同权重。

按照资源占用和依赖关系, ContextFlow 将一次上下文的生命周期概括为准备 (Prepare)、执行 (Execute) 和退休 (Retire) 三个阶段。准备阶段形成输入向量、权重和静态描述; 执行阶段独占 MAC 阵列; 退休阶段提交 PSUM 或最终输出并释放资源。相邻上下文可以同时处于不同阶段, 但同一时刻只有一个上下文取得阵列执行权。

4.3 执行路径与控制关系

片上向量生成与重放机制在第一次取得 HWC 分块时生成阵列向量, 并在多个输出块之间保持该序列, 对应式(3)的片外输入流量目标。面向折叠执行的上下文流水机制让下一上下文的准备和前一上

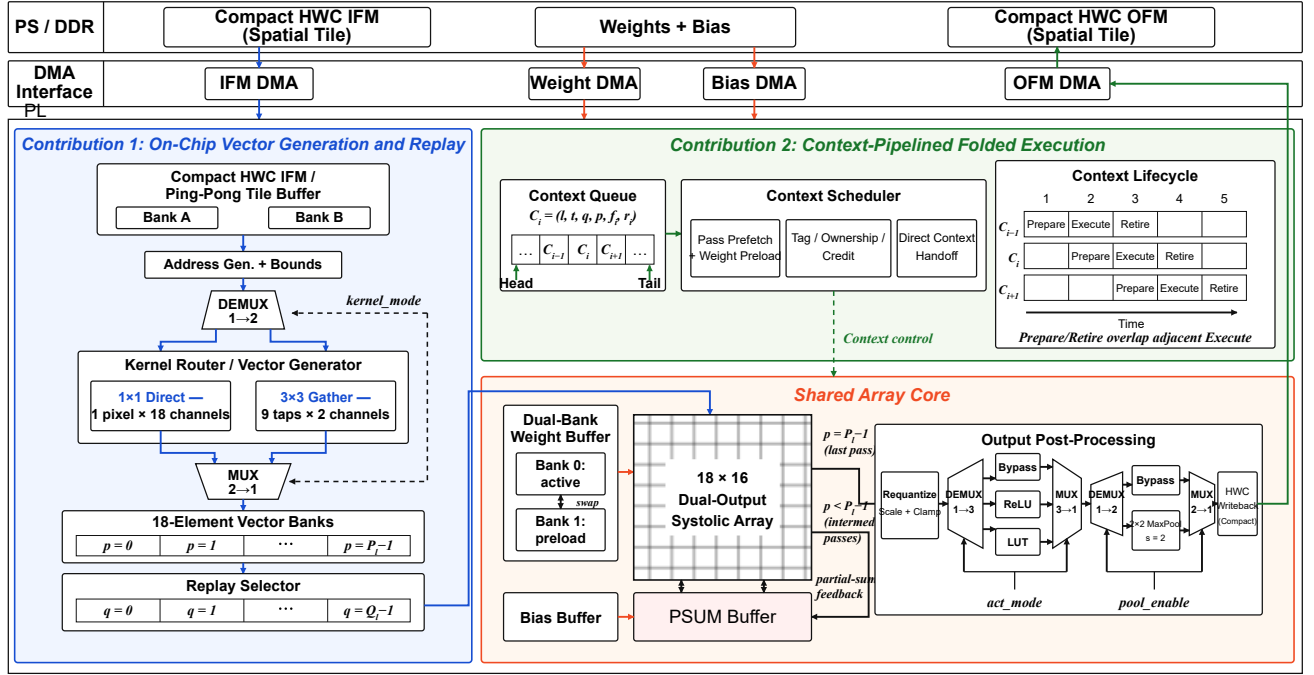


图 3 ContextFlow 总体架构与执行模型。外部保持紧凑 HWC 张量，片上向量重放组织跨输出块的数据复用，上下文流水组织相邻折叠执行的交接。

下文的退休与当前执行重叠，对应式(4)的边界缩短目标。第 6 节进一步说明数据依赖和资源安全对重叠时机的约束。

5 片上向量生成与重放

5.1 紧凑 HWC 输入与片上向量生成

ContextFlow 在系统边界保留紧凑 HWC 张量。PS 端无需生成显式的图像到列转换（image-to-column, im2col）中间张量，池化、上采样和分支拼接也可直接消费卷积输出。DMA 只搬运连续的紧凑数据，阵列所需的窗口展开在 PL 内部完成。

片上向量生成（on-chip vector generation）将紧凑 HWC 分块转换为阵列可直接消费的 18 元素输入向量序列，并把它保存在片上缓存中。3×3 路径使用三行缓存接收 HWC 字节；对输出位置 (y, x) 和第 p 轮计算，向量写为

$$v_{y,x,p}[r] = X[y + \Delta y(r, p), x + \Delta x(r, p), c(r, p)], \quad 0 \leq r < 18. \quad (6)$$

边界位置填入输入量化零值，末轮不足 18 个元素的

通路由有效位屏蔽。完整向量提交后才发布对应地址，因此反压只改变向量的可见时间，不改变提交粒度。

5.2 跨输出通道块重放

向量缓存按 (t, y, x, p) 寻址，地址中不含输出块下标 q 。同一空间分块的 $q = 0, 1, \dots, Q_t - 1$ 因而访问相同的向量序列，并配合各自的权重生成不同输出通道；这种复用称为片上向量重放（on-chip vector replay）。随 p 变化，不同轮次仍读取序列中各自对应的向量；重放发生在输出通道块之间，而不是把同一个向量用于不同输入片段。图 4 给出了相应的数据通路和复用边界。

向量缓存区间在最后一个输出块完成读取前保持有效，随后才允许下一空间分块覆盖。缓存容量因此只需覆盖当前活跃分块，而不必容纳完整网络张量。该生命周期把片外输入复用转化为有限片上容量和读带宽的权衡。

算法 1 给出上述机制的逻辑遍历顺序。空间分块进入片上后先形成 $V_t[p, i]$ ，其中 i 为分块内的输

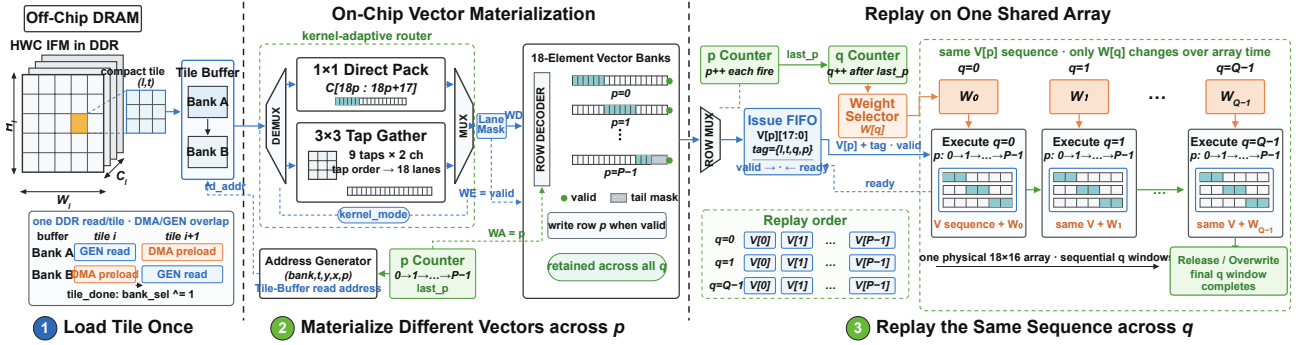


图 4 HWC 分块的片上向量生成与跨输出通道块重放机制。紧凑输入分块只读取一次；片上路由器按计算轮次 p 生成并缓存 18 元素向量，不同输出块 q 在同一物理阵列上重放整套向量序列，并在最后一个读取者完成后释放缓存。

算法 1: 片上向量生成与跨输出通道块重放的逻辑调度

输入: 空间分块集合 $\mathcal{T}$, 输出块数 Q_l , 计算轮数 P_l

输出: 按 HWC 顺序提交的输出分块

```

1  foreach 空间分块  $t \in \mathcal{T}$  do
2     $H_t \leftarrow \text{DDRRead}(t)$ 
3     $V_t[p, i] \leftarrow \text{Materialize}(H_t)$ 
4    for  $q \leftarrow 0$  至  $Q_l - 1$  do
5       $B_q \leftarrow \text{LoadBias}(q)$ 
6      for  $p \leftarrow 0$  至  $P_l - 1$  do
7         $W_{q,p} \leftarrow \text{LoadWeight}(q, p)$ 
8        foreach 分块内输出位置  $i$  do
9           $X \leftarrow \text{Replay}(V_t[p, i])$ 
10          $S \leftarrow B_q$  ( $p = 0$ ) 或  $\text{PSUM}_{p-1}[i]$  ( $p > 0$ )
11          $Y \leftarrow \text{MAC}_{32 \times 18}(X, W_{q,p}, S)$ 
12         if  $p = P_l - 1$  then
13           Emit(Pool(Act(Requant(Y))))
14         else
15           ForwardPSUM( $p, i, Y$ )
16       Release( $t$ )

```

出位置；该向量集合位于输出块循环之外，因此只生成一次。随后，每个输出块 q 依次遍历计算轮次 p ，首轮以偏置为初始部分和，后续轮次读取上一轮提交的 PSUM。

该伪代码只描述依赖关系和逻辑次序。实际硬件在当前轮次计算期间预取下一轮次的权重与输入

向量，并通过乒乓缓冲和基于信用的流控逐像素反馈 PSUM；因此，下一轮次不必等待整轮部分和全部写回后才启动。末轮不足 18 个乘加项或末个输出块不足 32 个通道时，分别由向量有效位和输出通道有效数屏蔽无效通路。

5.3 统一卷积接口与片外流量

1×1 路径不需要三行窗口，直接沿通道方向每次提取 18 个元素。它与 3×3 路径共享向量接口、上下文描述和重放地址，只改变 $(\Delta y, \Delta x, c)$ 的映射。原生路径避免把 1×1 权重填充为稀疏 3×3 核产生的结构性零计算，并减少不必要的核位置调度。

一次输入向量生成从片外读取约 $\sum_p B_{X,p} \approx B_X$ 字节的紧凑输入，后续向量写入和跨输出块读取都发生在片上。因此，该数据流实现了式(3)中 $D_{\text{replay}} \approx B_X$ 的目标。实际降低倍数还取决于空间分块、边界复用和 DMA 突发对齐；节 7 使用板端 DMA 字节计数而非理论重复因子报告最终结果。

6 面向折叠执行的上下文流水

6.1 从串行轮次到上下文流水

串行控制器完成 C_i 的全部准备、执行和退休后才开始 C_{i+1} 。ContextFlow 采用面向折叠执行的上下文流水机制 (context pipelining for folded execution): C_i 使用阵列时, C_{i+1} 可以准备地址与权重, C_{i-1} 则继续提交 PSUM 或最终结果。阵列仍只执行一个上下文, 但其前后阶段分布在相互独立

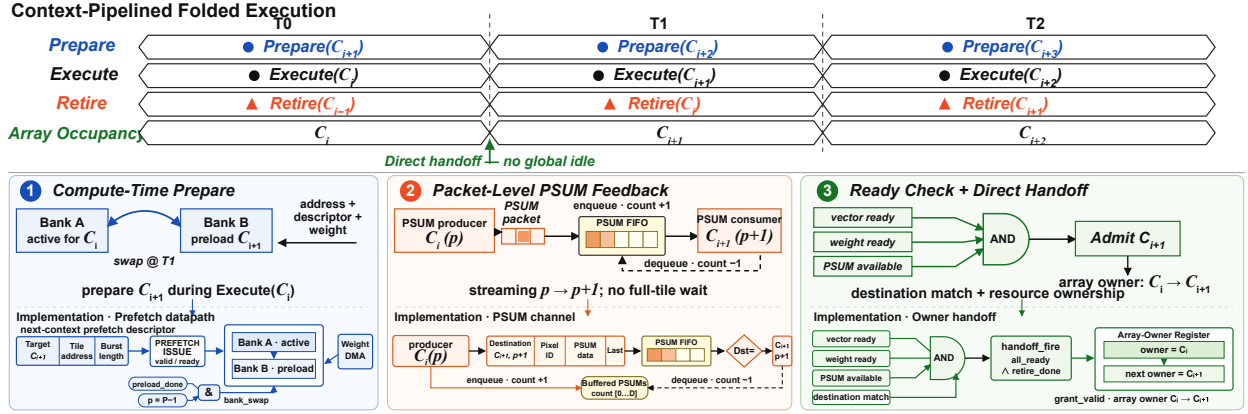


图5 面向折叠执行的上下文流水时序。相邻上下文的准备、计算和退休在阵列独占约束下重叠；计算期预取、数据包级部分和反馈及就绪检查支持直接切换。

的数据路径上，并按照图5所示时序重叠。

6.2 地址预取、权重预载与计算期准备

下一轮预取根据当前空间分块、输出块和轮次，提前生成下一上下文的地址、描述和有效通路掩码。权重存储采用当前槽和备用槽：当前槽供 C_i 使用，备用槽在当前计算期间接收 C_{i+1} 的权重；层、输出块和轮次身份匹配后，备用槽才在边界处成为当前槽。

下一轮预取决定下一上下文需要准备什么，计算期预取决定这些工作何时启动。ContextFlow 允许准备逻辑在当前上下文进行有效 MAC 发射时推进，把控制描述生成和部分数据准备移入当前计算区间。若准备持续时间超过当前计算，未覆盖部分仍会表现为边界等待。

6.3 数据包级部分和反馈与直接切换

第 p 轮对输出通道 c_o 的累加关系为

$$s_p[c_o] = s_{p-1}[c_o] + \sum_{r=0}^{17} v_p[r] w_p[c_o, r]. \quad (7)$$

首轮从偏置开始，末轮进入重定标和 HWC 写回。部分和反馈（partial-sum feedback）把非末轮结果按照相同像素和输出块身份送到下一计算轮次。传统分块级返回需要等待整个分块完成；ContextFlow 每提交一个完整像素数据包，下一轮即可消费该位置的 PSUM，从而把全分块屏障缩小为局部生产者–

消费者依赖。

当前上下文完成规定提交后，若下一上下文的输入、权重和必要 PSUM 均已就绪，则不经过全局空闲状态直接取得阵列执行权。这一过程称为上下文直接切换（direct context handoff）。权重预载与直接切换来自同一架构修改，因此实验只报告二者的组合增量收益。

6.4 重叠执行的正确性与稳态模型

重叠执行由三类条件保护。上下文身份中的代次区分循环复用缓存的新旧事务；资源所有权（resource ownership）把权重槽、向量缓存区间和 PSUM 槽绑定到当前使用者，并保持到最后一次读取或提交；信用控制（credit-based flow control）记录生产者已经完整提交的数据包数量，消费者只能读取可见范围内的 PSUM。外部输出仍按网络顺序退休。这些条件共同决定上下文何时能够安全接管资源。

片上向量生成与重放将重复输入合并为一次向量生成时间 T_V 。当下一权重、当前计算和当前退休使用独立资源时，流水稳态近似为

$$T_{pipe} \approx T_V + \sum_i \max(T_W^{i+1}, T_C^i, T_F^i) + T_{tail}. \quad (8)$$

性能由并行阶段中的最长项主导，而不再是所有阶段的简单累加。该模型不假设准备和退休开销总能被完全隐藏；它只描述从串行关系转为稳态重叠后

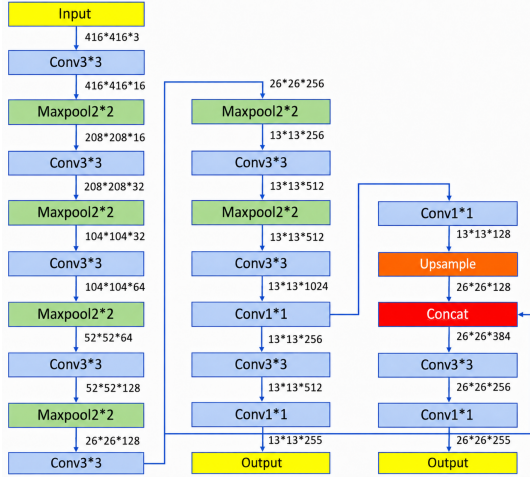


图 6 实验采用的双检测头 YOLOv3-tiny 网络拓扑。卷积层映射到 PL，池化、上采样、拼接和检测后处理由 PS 执行。

的性能边界。

7 实验评估

7.1 平台、负载与测量方法

实验平台为 AMD Xilinx KV260，器件为 XCK26。PL 运行频率为 200 MHz，使用 18×16 双输出阵列，共包含 576 个并行 MAC。四路 DMA 分别搬运偏置、权重、IFM 和 OFM。完整负载为输入尺寸 416×416 的双检测头 YOLOv3-tiny，13 个卷积层全部映射到 PL；池化、最近邻上采样、分支拼接和检测后处理由四核 Cortex-A53 执行。后处理包含候选框解码和 NMS。图 6 给出实验负载中的主要张量尺寸、双尺度检测头以及分支拼接关系。

部署模型先使用 1024 张图像进行直方图标定 (PTQ)；冻结量化参数后，仅对 13 个已融合 BatchNorm 的卷积层权重和偏置进行量化感知训练 (QAT) 微调。

常驻推理延迟从图像进入网络调度开始，到双检测头完成 NMS 为止，模型参数在连续图像之间保持驻留；PL 延迟只统计 13 个卷积。吞吐率以 FPS 表示，有效计算性能以 GOPS 表示，并按一次乘法和一次加法计为两个操作。有效阵列利用率由完整网络有效卷积操作量、PL 时间、576 个 MAC 和 200 MHz 峰值共同计算。

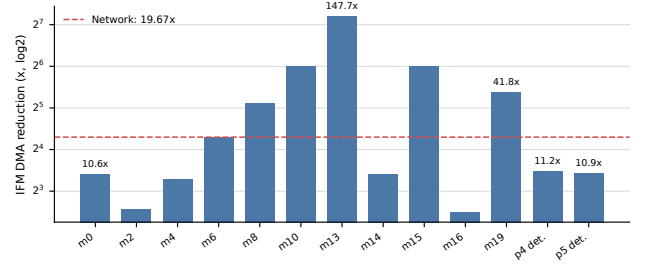


图 7 片上向量生成与重放相对串行输入方式的逐层 IFM DMA 降低倍数。纵轴采用以 2 为底的对数刻度，虚线表示完整网络的 19.67 倍降低。

五级消融依次为串行基线、片上向量生成与重放、下一轮预取、权重预载与直接切换，以及计算期预取。每项结果来自 3 个独立会话；每个会话预热 20 张图像，再计时 1000 张，总计 3000 个样本。下文以 P95 描述时延分布尾部。

7.2 片外输入流量

串行基线与向量生成重放配置处理相同的 13 层卷积和 3000 张计时图像，权重与输出字节数保持不变。完整网络 IFM DMA 字节由 53,609,504 降至 2,725,632，降低 19.67 倍，说明输入搬运已经由逐折叠执行转化为逐空间分块。

图 7 表明实际降低倍数也不简单等于 Q_l 。m13 和 m19 具有较多输出通道块与较大的输入重放空间；早期层更容易受空间分块数量、边界行复用和 DMA 突发对齐影响。向量生成与重放同时使常驻推理延迟由 156.457 ms 降至 77.445 ms，获得 2.020 倍加速，说明流量降低带来了完整网络加速。

7.3 上下文流水的持续利用率

图 8 给出五级消融。下一轮预取使延迟由 77.445 ms 降至 63.451 ms，获得 1.221 倍相邻加速；权重预载与直接切换把延迟进一步降至 40.025 ms，组合收益为 1.585 倍；加入计算期预取后，延迟达到 34.943 ms，对应 1.145 倍加速。

完整数据集上的模型精度汇总于表 4。

有效阵列利用率从串行基线的 20.14% 依次提高到 31.75%、38.91%、62.48% 和 71.87%。所有配置使用相同阵列规模、频率和有效网络 MAC 数，因

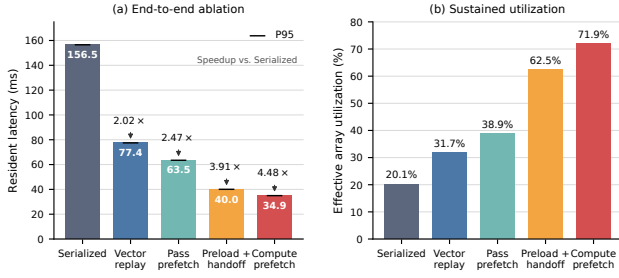


图 8 完整网络五级消融。(a) 柱高为常驻推理延迟均值，黑色短横线为 P95；(b) 为相同配置的有效阵列利用率。

表 4 COCO val2017 上的模型精度

模型	图像数	AP/AP50
FP32-416	5000	17.494%/33.493%
INT8-416	5000	14.304%/30.212%

此延迟下降与利用率上升共同反映了输入搬运和边界等待的减少。该消融反映轮次边界优化的综合效果，不对应式(4)中的气泡周期计数。

7.4 逐层收益与卷积形状

图 9 表明最终配置加速了全部 13 个卷积，但收益随层形状变化。m13 和 m19 的累计加速分别约为 4.39 倍和 3.95 倍：较大的 Q_i 带来更多跨输出块输入复用，较大的 $P_i Q_i$ 则形成更长的上下文流水稳态区。m16 只有 15 轮计算和 4 个输出块，累计加速约 1.46 倍，准备阶段在总时间中的比例较小。空间尺寸、分块数量、权重流量和尾块也会影响最终收益。

7.5 端到端性能与实现成本

最终配置的常驻推理延迟均值为 34.943 ms，对应 28.618 FPS；13 层 PL 卷积延迟为 33.607 ms，对应 165.588 GOPS 和 71.87% 的有效阵列利用率。PS 上的张量变换与检测后处理合计 1.301 ms。

实现使用 576 个阵列 DSP，全系统共 650 个 DSP。48 个 URAM 主要保存生成后的输入向量，BRAM 保存行缓存、权重与 PSUM。CLB 占用率为 76.13%。WNS 为 0.004 ns，说明 200 MHz 已接近当前布局布线边界。4.008 W 仅作为布局布线后的无向量功耗估算（post-route vectorless estimate）。

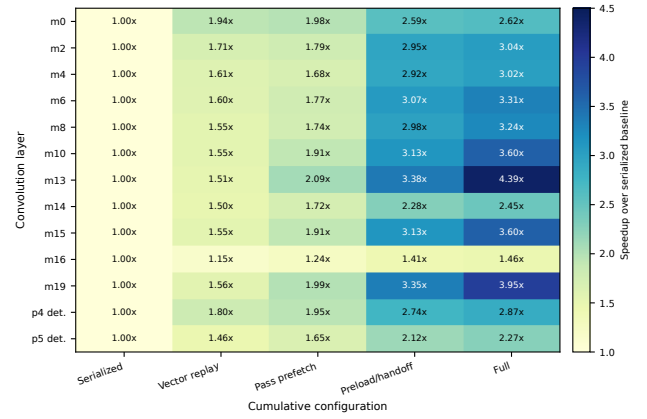


图 9 五级配置相对串行基线的逐层累计加速。每个单元格为 $T_{\text{serialized}}/T_{\text{configuration}}$ ，数值越大表示累计收益越高。

表 5 正式冻结运行的端到端性能与实现代价。功耗为布局布线后工具估算。

性能指标	数值	实现指标	数值
Resident 均值	34.943 ms	LUT	56,949
Resident P95	34.965 ms	DSP	650
PL 卷积	33.607 ms	阵列 DSP	576
吞吐率	28.618 FPS	BRAM	94
有效吞吐	165.588 GOPS	URAM	48
有效阵列利用率	71.87%	CLB 占用	76.13%
WNS	0.004 ns	估算功耗	4.008 W

7.6 重叠执行的整数正确性

正确性覆盖三个层次：128 张图像的 22 个整数节点逐字节比较、128 张图像的两个原始检测头比较，以及 5000 张 COCO val2017 图像的检测指标比较。模型精度以 AP 表示，并以 FP32 模型作为参考；完整数据集的 FP32 与 INT8 精度列于表 4。

板端共形成 2,816 条逐节点记录，所有整数节点和原始检测头均与整数参考逐字节一致，完整数据集指标也保持一致。FP32 与 INT8 的 AP 差异来自当前量化参数：FP32-416 的 AP/AP50 为 17.494%/33.493%，INT8-416 为 14.304%/30.212%。逐节点零失配则说明片上重放和上下文重叠没有改变硬件整数语义。

8 讨论

8.1 适用范围与收益边界

CONTEXTFLOW 的收益与 P_l 、 Q_l 共同相关。 Q_l 决定同一输入分块能够覆盖的输出通道块数量, P_l 表示完成一次输出所需的计算轮数。二者较大时, 串行执行既会重复读取更多输入, 也会暴露更多轮次边界; 同时, 较长的计算阶段为下一上下文的准备提供了更充分的覆盖窗口。因此, m13 和 m19 这类深层标准卷积取得了最高的逐层累计加速。相反, 当 P_l 或 Q_l 较小时, 输入重复和边界数量本就有限, 短上下文也难以形成长稳态区。m16 的累计加速约为 1.46 倍, 反映的正是这一边界。

这也说明 ContextFlow 并非面向所有卷积形状提供等幅收益。其主要适用对象是固定规模阵列上的深层折叠卷积; 对于单轮即可完成的浅层、深度卷积或分组卷积, 需要依据其通道复用关系重新组织向量和上下文遍历。

8.2 片上存储与时序代价

减少片外输入并不意味着数据移动成本消失, 而是将其中一部分转化为片上缓存容量、端口带宽与布线压力。当前实现使用 48 个 URAM 保存生成后的输入向量, 占器件 URAM 的 75%; 200 MHz 下 WNS 为 0.004 ns, 表明时序余量已经较小。扩大空间分块通常能增加重放次数, 却会同步增加向量缓存容量和跨阶段连接。因此, 空间分块、缓存规模与目标频率应结合器件资源共同选择, 而不能单独追求更高的理论复用率。

8.3 扩展到更大系统

在单阵列实现中, 向量缓存只需服务一个主要消费者。扩展到多阵列后, 同一输入序列可能面对多个并发读者, 瓶颈将从片外带宽进一步转向片上重放带宽和缓存端口数。类似地, 更大的阵列可以减少 P_l 或 Q_l , 却也可能缩短流水的稳态区。因而, ContextFlow 的扩展不只是复制计算单元, 还需要联合调整阵列规模、上下文粒度和向量缓存组织。

Reported Resource Efficiency

Reported throughput normalized by total accelerator resources

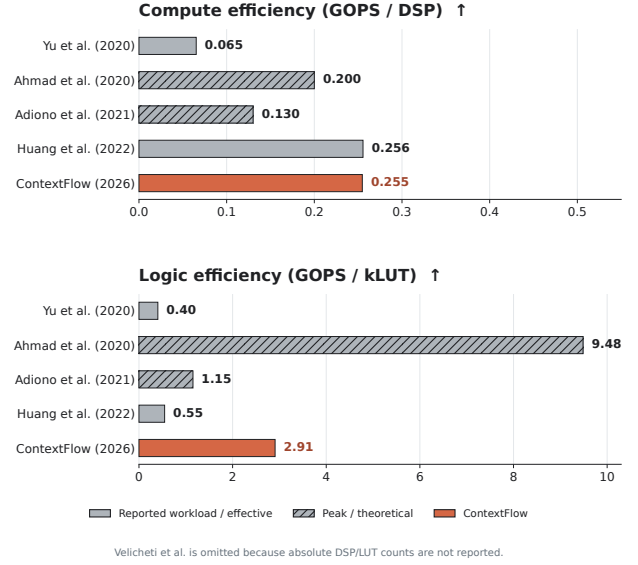


图 10 公开工作的资源归一化吞吐^[11-12,6-7]。计算效率按 GOPS/DSP 表示, 逻辑效率按 GOPS/kLUT 表示; 斜线条为峰值或理论口径, 实心条为完整负载或有效口径。

8.4 机制边界

CONTEXTFLOW 不改变卷积算术、模型量化参数或峰值 MAC 数, 而是在紧凑张量接口和折叠阵列之间重新组织数据复用与执行交接。IFM DMA 流量用于刻画片上重放效果, 消融延迟和有效阵列利用率用于刻画流水后的综合变化。因此, 本文报告的收益来自持续供数和边界调度, 而不是网络计算量或计算单元数量的变化。

9 相关工作

9.1 数据流、折叠与命令重叠

Eyeriss 和 Timeloop 分别从数据移动代价与映射空间出发, 说明循环次序和存储层次如何决定输入、权重与部分和复用^[4,13]。Angel-Eye 进一步把 IFM 分块驻留与 LOAD/CALC/SAVE 依赖结合, 用编译调度减少片外搬运并重叠数据访问^[5]; Adiono 等则在硬件中生成压缩 im2col 表示, 避免完整展开张量^[6]。

Huang 等使用循环分块、乒乓传输和独立 PSUM 缓冲支持固定阵列上的跨输入块累加^[7];

Systolic-CNN 通过输入缓存、PE 内累加和深流水适配不同长度的卷积^[8]；Gemmini 以解耦访存/执行队列和跨命令依赖组织矩阵块计算^[9]。这些机制构成 ContextFlow 的基础。本文进一步把调度单位明确为一次 (l, t, q, p) 折叠执行，使按 p 生成的输入向量跨 q 重放，并使准备、执行与退休跨相邻上下文重叠。

9.2 FPGA YOLO 加速器

Adiono、Velicheti 和 Yu 等分别以输入驻留矩阵乘、一维脉动阵列和参数化架构实现 YOLOv3-tiny^[6,14,11]。不同器件、位宽和计时边界使结果不能直接等价比较；本文以 GOP 表示单图算术量，并仅将公开结果用于系统级资源效率定位。

图 10 按公开的总 DSP 和 LUT 数量归一化吞吐，以区分峰值算力与完整负载的有效吞吐。斜线条为峰值或理论值，实心条为完整负载报告值；该图仅用于定位资源效率，不构成跨模型、器件和精度的等价比较。

9.3 折叠执行的组织粒度

表 6 主要参考工作与 ContextFlow 的数据对象及执行组织粒度

工作	数据对象、执行粒度及与本文问题的关系
Angel-Eye ^[5]	IFM/OFM 分块；LOAD/CALC/SAVE 指令；输入驻留与命令重叠。
Adiono 等 ^[6]	压缩 im2col 分块；层内循环与输入驻留；片上表示转换。
Huang 等 ^[7]	输入、权重与 PSUM 分块；循环分块与乒乓传输；固定阵列折叠累加。
Gemmini ^[9]	暂存区矩阵块；解耦访存/执行命令；跨命令依赖与异步执行。
ContextFlow	按 p 组织输入向量； (l, t, q, p) 折叠上下文；跨 q 重放与相邻上下文交接。

表 6 按数据对象和执行粒度归纳了上述差异。已有方案分别覆盖分块驻留、片上表示转换、跨输入块累加以及访存与执行解耦；CONTEXTFLOW 则将一次 (l, t, q, p) 折叠执行定义为基本上下文，使按 p 生成的输入向量跨 q 重放，并连续衔接相邻上下文的准备、执行和退休。数据复用与阵列交接由此在现有片上存储组织中采用同一执行粒度。

10 结论

固定规模阵列执行深层卷积时，折叠映射会形成由多轮计算和多个输出通道块构成的执行序列。如果数据复用与执行交接仍停留在较粗粒度，同一输入分块将从 DDR 重复读入片上，准备和提交阶段造成的上下文边界气泡也会随执行序列累积。CONTEXTFLOW 将一次折叠执行作为共同的组织单位：片上向量生成与重放把紧凑 HWC 分块转换为可跨输出块共享的阵列输入向量，面向折叠执行的上下文流水则在依赖约束下并行推进相邻上下文的准备、计算、部分和反馈与结果提交。

输入流量、完整网络消融、逐层延迟和端到端结果表明，上述组织方式能够提高固定阵列的持续供数能力和有效阵列利用率；板端节点比对同时确认重放与重叠没有改变整数计算结果。对于需要多轮折叠的量化卷积，片上数据表示及其有效范围应与轮次间调度共同设计，使算法中的复用机会能够落实为连续的阵列执行。

参考文献

- [1] KUNG H T. Why systolic architectures?[J/OL]. Computer, 1982, 15(1): 37-46. DOI: 10.1109/MC.1982.1653825.
- [2] ZHANG C, LI P, SUN G, et al. Optimizing fpga-based accelerator design for deep convolutional neural networks[C/OL]//ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2015: 161-170. DOI: 10.1145/2684746.2689060.
- [3] QIU J, WANG J, YAO S, et al. Going deeper with embedded fpga platform for convolutional neural network[C/OL]//ACM/SIGDA International Symposium on Field-Programmable Gate Arrays. 2016: 26-35. DOI: 10.1145/2847263.2847265.
- [4] CHEN Y H, EMER J, SZE V. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks[C/OL]//43rd Annual International Symposium on Computer Architecture. 2016: 367-379. DOI: 10.1109/ISCA.2016.40.
- [5] GUO K, SUI L, QIU J, et al. Angel-Eye: A complete design flow for mapping cnn onto embedded fpga[J/OL]. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2018, 37(1): 35-47. DOI: 10.1109/TCAD.2017.2705069.
- [6] ADIONO T, PUTRA A, SUTISNA N, et al. Low latency YOLOv3-Tiny accelerator for low-cost fpga using general matrix multiplication principle[J/OL]. IEEE Access, 2021, 9: 141890-141913. DOI: 10.1109/ACCESS.2021.3120629.

-
- [7] HUANG H, HU X, LI X, et al. An efficient loop tiling framework for convolutional neural network inference accelerators[J/OL]. IET Circuits, Devices and Systems, 2022, 16(1): 116-123. DOI: 10.1049/cds2.12091.
- [8] DUA A, LI Y, REN F. Systolic-cnn: An opencl-defined scalable runtime-flexible fpga accelerator architecture for accelerating convolutional neural network inference in cloud/edge computing[C/OL]//2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines. 2020: 231-231. DOI: 10.1109/FCCM48280.2020.00064.
- [9] GENC H, KIM S, AMID A, et al. Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration[C/OL]//58th ACM/IEEE Design Automation Conference. 2021: 769-774. DOI: 10.1109/DAC18074.2021.9586216.
- [10] SZE V, CHEN Y H, YANG T J, et al. Efficient processing of deep neural networks: A tutorial and survey[J/OL]. Proceedings of the IEEE, 2017, 105(12): 2295-2329. DOI: 10.1109/JPROC.2017.2761740.
- [11] YU Z, BOUGANIS C S. A parameterisable fpga-tailored architecture for YOLOv3-Tiny[C/OL]//Lecture Notes in Computer Science: Vol. 12083 Applied Reconfigurable Computing. Architectures, Tools, and Applications. 2020: 330-344. DOI: 10.1007/978-3-030-44534-8_25.
- [12] AHMAD A, PASHA M A, RAZA G J. Accelerating tiny YOLOv3 using FPGA-based hardware/software co-design[C/OL]//2020 IEEE International Symposium on Circuits and Systems (ISCAS). 2020: 1-5. DOI: 10.1109/ISCAS45731.2020.9180843.
- [13] PARASHAR A, et al. Timeloop: A systematic approach to dnn accelerator evaluation[C/OL]//IEEE International Symposium on Performance Analysis of Systems and Software. 2019: 304-315. DOI: 10.1109/ISPASS.2019.00042.
- [14] VELICHETI P, PENTAPATI S, PURINI S. Systolic array based fpga accelerator for YOLOv3-Tiny[C/OL]//IEEE High Performance Extreme Computing Conference. 2022. DOI: 10.1109/HPEC55821.2022.9926371.